

Hospital Patient & Appointment Management System

A Comprehensive Technical Report on Relational Data Modeling, Normalization, Integrity Constraints, Advanced SQL Operations, and Transaction Safety with Savepoints.

AUTHOR	DATABASE ENGINE	DATABASE NAME	TOTAL MARKS
Aaryan Keni	PostgreSQL 18.4 (Homebrew)	hospitaldb	20 / 20

1. Executive Summary & System Scope

Modern hospital information management mandates high reliability, strict temporal scheduling controls, and flexible clinical data structures. This engineering project implements a robust relational database in **PostgreSQL** supporting multi-specialty hospital operations:

- **Departmental Organization:** Grouping doctors under medical disciplines (Cardiology, Orthopedics, Pediatrics).
- **Doctor Profiles & Pricing:** Managing physician specializations and distinct departmental consultation fees.
- **Patient Registry & Semi-Structured EHR:** Using PostgreSQL's high-performance `JSONB` type to capture nested medical history (allergies, blood groups, and emergency contacts) without brittle relational schema migrations.
- **Doctor Scheduling:** Strictly enforcing temporal conflict constraints through a composite candidate key to guarantee that a doctor is never double-booked at the same date and time.
- **Clinical Prescription & Transaction Safety:** Demonstrating ACID atomicity and granular rollback through SQL `SAVEPOINT`s to prevent allergic medication errors while preserving scheduled consultations.

2. Part A: ER Model & Database Design

5 Marks

The system is designed around five core entities: `Department`, `Doctor`, `Patient`, `Appointment`, and `Prescription`.

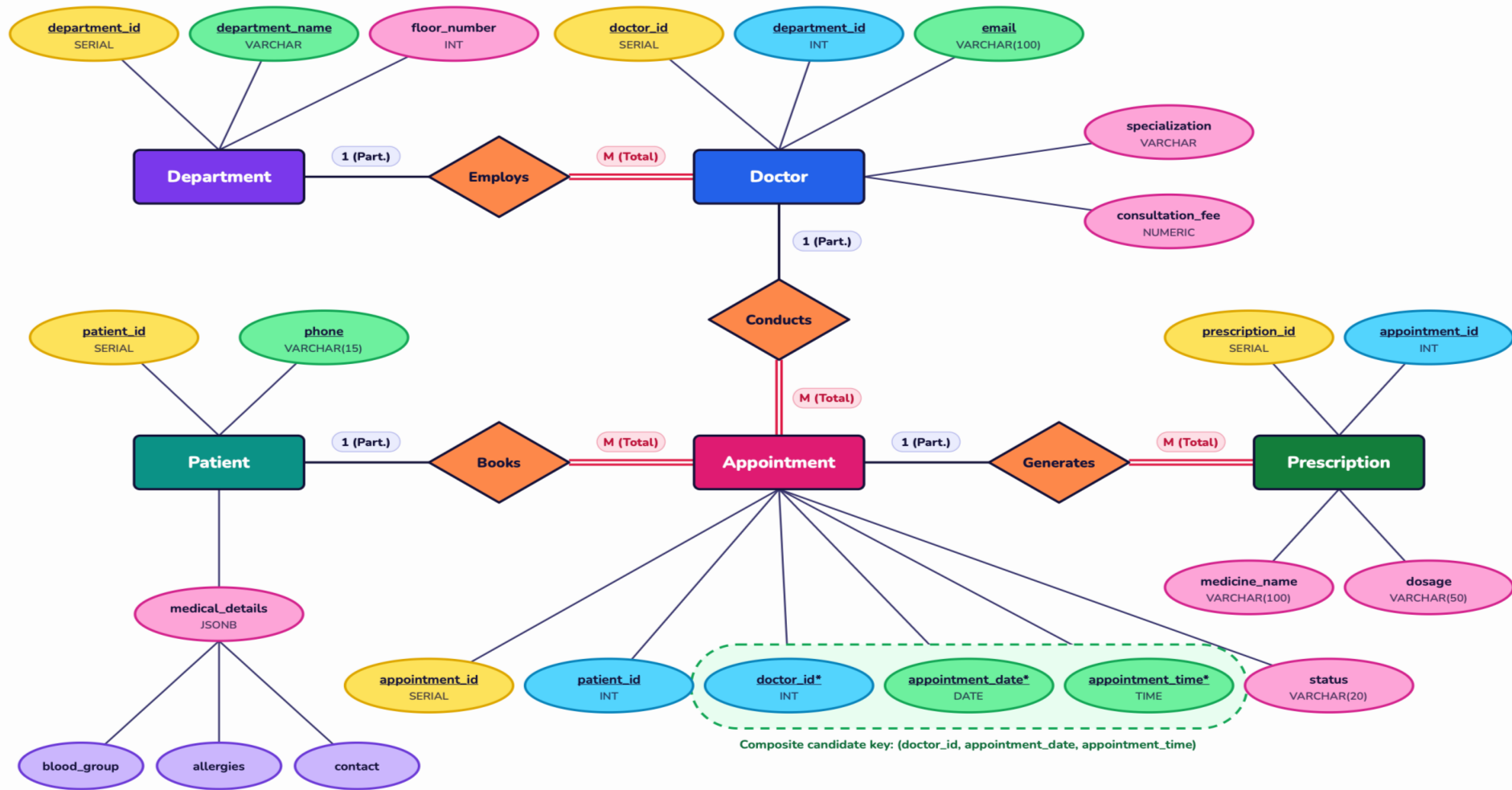
Entity-Relationship Specifications

RELATIONSHIP	ENTITIES INVOLVED	CARDINALITY	PARTICIPATION	BUSINESS RULE & INTEGRITY RATIONALE
Employs / Has	Department ↔ Doctor	1 : M	Doctor: Total Department: Partial	Every doctor must belong to exactly one department (<code>department_id NOT NULL</code>). A department may exist before doctors are recruited.
Conducts / Consults	Doctor ↔ Appointment	1 : M	Appointment: Total Doctor: Partial	An appointment must be assigned to a licensed doctor. A doctor can exist without currently having any scheduled appointments.

Books	Patient ↔ Appointment	1 : M	Appointment: Total Patient: Partial	Every appointment belongs to exactly one patient. A registered patient may exist without an active appointment on file.
Generates / Prescribes	Appointment ↔ Prescription	1 : M	Prescription: Total Appointment: Partial	A prescription line item must link to an appointment consultation. An appointment may conclude without issuing any prescription.

Entity-Relationship (ER) Diagram

Below is the high-fidelity ER Diagram illustrating entities, attributes, primary/foreign keys, candidate keys, cardinalities, and participation semantics (single line for partial, double line for total participation).



Legend

Primary key (underlined)

Foreign key (dashed)

Candidate key (dotted)

Attribute

Sub-attribute of a composite

Entity

Relationship

Partial participation (0..M, optional)

Total participation (mandatory, 1..M)

* Part of composite key

3. Part B: Relational Schema & Keys Analysis

5 Marks

To ensure relational integrity and eliminate anomalies (meeting Third Normal Form - 3NF), key constraints are systematically established across all relations.

Systematic Key Identification Matrix

TABLE NAME	PRIMARY KEY (PK)	ALTERNATE CANDIDATE KEY (CK)	FOREIGN KEYS (FK) & REFERENCES	COMPOSITE KEY
Departments	department_id (SERIAL)	department_name (UNIQUE)	None	None
Doctors	doctor_id (SERIAL)	email (UNIQUE)	department_id → Departments(department_id)	None
Patients	patient_id (SERIAL)	phone (UNIQUE)	None	None
Appointments	appointment_id (SERIAL)	(doctor_id, appointment_date, appointment_time)	patient_id → Patients(patient_id) doctor_id → Doctors(doctor_id)	UNIQUE(doctor_id, appointment_date, appointment_time)
Prescriptions	prescription_id		appointment_id →	None

(SERIAL)

(appointment_id,
medicine_name)

Appointments(appointment_id)

⚠ Real-World Rule Enforced by Composite Candidate Key

Constraint: UNIQUE (doctor_id, appointment_date, appointment_time) on table Appointments .

Clinical / Operational Rule: *Physician Double-Booking Prevention*. In a hospital, a single medical practitioner can only be physically present in one consultation room at any given time slot. This composite unique index guarantees at the database storage engine layer that no race condition or user error can schedule two distinct patients with the same doctor at an overlapping date and time.

4. Database Implementation & Execution Proof

The schema was created inside PostgreSQL database `hospitaldb` . The following terminal captures document the DDL creation and initial data population.

4.1 Environment & Table DDL Creation (Screenshots 1 - 6)

```
[kunal@kunal ~] v22.23.2 06:03 > psql postgres
psql (18.4 (Homebrew))
Type "help" for help.
```

```
[postgres=# CREATE DATABASE hospitaldb;
CREATE DATABASE
[postgres=# \c hospitaldb
You are now connected to database "hospitaldb" as user "kunallubhana".
[hospitaldb=# \dt
Did not find any tables.
hospitaldb=#
```

Screenshot 1: psql session initialization in hospitaldb.

```
Did not find any tables.
hospitaldb=# CREATE TABLE departments (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(100) UNIQUE NOT NULL,
    floor_number INT
);
CREATE TABLE
hospitaldb=#
```

Screenshot 2: DDL execution for `departments` .

```
hospitaldb=# CREATE TABLE doctors (  
    doctor_id SERIAL PRIMARY KEY,  
    department_id INT NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    specialization VARCHAR(100),  
    consultation_fee NUMERIC(8,2),  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
[);  
CREATE TABLE  
hospitaldb=#
```

Screenshot 3: DDL execution for `doctors` with foreign key to departments.

```
hospitaldb=# CREATE TABLE patients (  
    patient_id SERIAL PRIMARY KEY,  
    phone VARCHAR(15) UNIQUE NOT NULL,
```

```
        medical_details JSONB
    );
CREATE TABLE
hospitaldb=#
```

Screenshot 4: DDL execution for `patients` with JSONB `medical_details`.

```
hospitaldb=# CREATE TABLE appointments (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status VARCHAR(20),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
    UNIQUE (doctor_id, appointment_date, appointment_time)
);
CREATE TABLE
hospitaldb=#
```

Screenshot 5: DDL for `appointments` with foreign keys and composite unique constraint.

```
hospitaldb=# CREATE TABLE prescriptions (  
    prescription_id SERIAL PRIMARY KEY,  
    appointment_id INT NOT NULL,  
    medicine_name VARCHAR(100) NOT NULL,  
    dosage VARCHAR(50),  
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)  
[ );  
CREATE TABLE  
hospitaldb=#
```

Screenshot 6: DDL execution for `prescriptions` linked to appointments.

4.2 Data Ingestion & Seeding Verification (Screenshots 7 - 12)

```
hospitaldb=# INSERT INTO departments (department_id, department_name, floor_number)  
VALUES  
(1, 'Cardiology', 2),  
(2, 'Orthopedics', 3),  
[(3, 'Pediatrics', 1);  
INSERT 0 3
```

```

[hospitaldb=# SELECT * FROM departments;
 department_id | department_name | floor_number
-----+-----+-----
            1 | Cardiology      |            2
            2 | Orthopedics     |            3
            3 | Pediatrics      |            1
(3 rows)

hospitaldb=#

```

Screenshot 7: Ingestion of Cardiology, Orthopedics, and Pediatrics.

```

hospitaldb=# INSERT INTO doctors
(doctor_id, department_id, email, specialization, consultation_fee)
VALUES
(1, 1, 'asha.rao@hosp.com', 'Cardiologist', 800),
(2, 1, 'vikram.singh@hosp.com', 'Cardiologist', 650),
(3, 2, 'neha.jain@hosp.com', 'Orthopedic Surgeon', 700),
(4, 2, 'rohit.mehta@hosp.com', 'Orthopedic Surgeon', 550),
(5, 3, 'priya.desai@hosp.com', 'Pediatrician', 500);
INSERT 0 5
[hospitaldb=# SELECT * FROM doctors;
 doctor_id | department_id | email | specialization | consultation_fee
-----+-----+-----+-----+-----
            1 |            1 | asha.rao@hosp.com | Cardiologist |            800.00
            2 |            1 | vikram.singh@hosp.com | Cardiologist |            650.00
            3 |            2 | neha.jain@hosp.com | Orthopedic Surgeon |            700.00
            4 |            2 | rohit.mehta@hosp.com | Orthopedic Surgeon |            550.00
            5 |            3 | priya.desai@hosp.com | Pediatrician |            500.00
(5 rows)

```

Screenshot 8: Ingestion of 5 doctors with specializations and fees.

Screenshot 9: Ingestion of 3 patients with JSONB allergy and emergency contact data.

```
hospitaldb=# INSERT INTO appointments
(appointment_id, patient_id, doctor_id, appointment_date, appointment_time, status)
VALUES
```

```
VALUES
(1, 1, 1, '2026-09-20', '10:00', 'Scheduled'),
(2, 2, 3, '2026-09-20', '11:30', 'Scheduled'),
(3, 1, 1, '2026-09-21', '09:00', 'Completed'),
(4, 3, 5, '2026-09-22', '14:00', 'Scheduled');
INSERT 0 4
hospitaldb=# SELECT * FROM appointments;
 appointment_id | patient_id | doctor_id | appointment_date | appointment_time | status
-----+-----+-----+-----+-----+-----
              1 |          1 |          1 | 2026-09-20      | 10:00:00        | Scheduled
              2 |          2 |          3 | 2026-09-20      | 11:30:00        | Scheduled
              3 |          1 |          1 | 2026-09-21      | 09:00:00        | Completed
              4 |          3 |          5 | 2026-09-22      | 14:00:00        | Scheduled
(4 rows)

hospitaldb=#
```

Screenshot 10: Ingestion of appointments across dates and statuses.

```
hospitaldb=# INSERT INTO prescriptions
(prescription_id, appointment_id, medicine_name, dosage)
VALUES
(1, 3, 'Amoxicillin', '500mg twice daily'),
(2, 3, 'Paracetamol', '650mg as needed');
INSERT 0 2
hospitaldb=# SELECT * FROM prescriptions;
 prescription_id | appointment_id | medicine_name | dosage
-----+-----+-----+-----
              1 |              3 | Amoxicillin   | 500mg twice daily
              2 |              3 | Paracetamol   | 650mg as needed
(2 rows)
```

hospitaldb=#

Screenshot 11: Ingestion of Amoxicillin and Paracetamol for appointment 3.

```
[hospitaldb=# SELECT * FROM departments;
 department_id | department_name | floor_number
-----+-----+-----
          1 | Cardiology      |           2
          2 | Orthopedics     |           3
          3 | Pediatrics      |           1
(3 rows)

[hospitaldb=# SELECT * FROM doctors;
 doctor_id | department_id | email              | specialization | consultation_fee
-----+-----+-----+-----+-----
          1 |          1 | asha.rao@hosp.com | Cardiologist   |          800.00
          2 |          1 | vikram.singh@hosp.com | Cardiologist   |          650.00
          3 |          2 | neha.jain@hosp.com | Orthopedic Surgeon |          700.00
          4 |          2 | rohit.mehta@hosp.com | Orthopedic Surgeon |          550.00
          5 |          3 | priya.desai@hosp.com | Pediatrician    |          500.00
(5 rows)

[hospitaldb=# SELECT * FROM patients;
 patient_id | phone | medical_details
-----+-----+-----
          1 | 9876543210 | {"allergies": ["penicillin", "peanuts"], "blood_group": "O+", "emergency_contact": {"name": "Ramesh Kumar", "phone": "9876543210"}}
          2 | 9123456780 | {"allergies": ["dust"], "blood_group": "B+", "emergency_contact": {"name": "Sunita Kumar", "phone": "9123456780"}}
          3 | 9988776655 | {"allergies": [], "blood_group": "A+", "emergency_contact": {"name": "Amit Shah", "phone": "9988776655"}}
(3 rows)

[hospitaldb=# SELECT * FROM appointments;
 appointment_id | patient_id | doctor_id | appointment_date | appointment_time | status
-----+-----+-----+-----+-----+-----
              1 |          1 |          1 | 2026-09-20       | 10:00:00         | Scheduled
              2 |          2 |          3 | 2026-09-20       | 11:30:00         | Scheduled
              3 |          1 |          1 | 2026-09-21       | 09:00:00         | Completed
              4 |          3 |          5 | 2026-09-22       | 14:00:00         | Scheduled
(4 rows)

[hospitaldb=# SELECT * FROM prescriptions;
 prescription_id | appointment_id | medicine_name | dosage
-----+-----+-----+-----
              1 |              3 | Amoxicillin   | 500mg twice daily
              2 |              3 | Paracetamol   | 650mg as needed
(2 rows)
```

```
hospitaldb=#
```

Screenshot 12: Comprehensive multi-table SELECT verification across all 5 relations.

5. Part C: Advanced SQL Queries & Clinical Analytics

10 Marks

Q1. Correlated Subquery: Doctors Above Department Average Fee

Objective: Identify doctors who charge higher consultation fees than the average fee of their specific department.

```
SELECT
    d.doctor_id,
    d.department_id,
    d.email,
    d.specialization,
    d.consultation_fee
FROM doctors d
WHERE d.consultation_fee > (
    SELECT AVG(d2.consultation_fee)
    FROM doctors d2
    WHERE d2.department_id = d.department_id
);
```

Analytical Breakdown

- **Department 1 (Cardiology):** Dr. Asha Rao (800) and Dr. Vikram Singh (650). Department Average = $(800 + 650) / 2 = 725.00$. Dr. Asha Rao ($800 > 725$) qualifies.
- **Department 2 (Orthopedics):** Dr. Neha Jain (700) and Dr. Rohit Mehta (550). Department Average = $(700 + 550) / 2 = 625.00$. Dr. Neha Jain ($700 > 625$) qualifies.
- **Department 3 (Pediatrics):** Dr. Priya Desai (500). Department Average = **500.00**. No doctor is strictly greater than the average.

```

hospitaldb=# SELECT
    d.doctor_id,
    d.department_id,
    d.email,
    d.specialization,
    d.consultation_fee
FROM doctors d
WHERE d.consultation_fee >
(
    SELECT AVG(d2.consultation_fee)
    FROM doctors d2
    WHERE d2.department_id = d.department_id
);

```

doctor_id	department_id	email	specialization	consultation_fee
1	1	asha.rao@hosp.com	Cardiologist	800.00
3	2	neha.jain@hosp.com	Orthopedic Surgeon	700.00

(2 rows)

```

hospitaldb=#

```

Screenshot 13: Execution and query output for Q1 correlated subquery returning Asha Rao and Neha Jain.

Q2. Left Outer Join: Doctor Appointment Counts (Including Zero Appointments)

Objective: List all doctors alongside their appointment count, ensuring doctors without appointments are preserved with a count of 0.

```
SELECT
    d.doctor_id,
    d.email,
    COUNT(a.appointment_id) AS appointment_count
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.email
ORDER BY d.doctor_id;
```

Engineering Note: COUNT(column) vs COUNT(*)

Using `COUNT(a.appointment_id)` correctly evaluates to **0** when a doctor has no appointments, because `LEFT JOIN` produces a `NULL` row for unmatched appointments, and aggregate `COUNT(expr)` ignores NULLs. In contrast, `COUNT(*)` would incorrectly report 1 appointment for unmatched doctors.

```

hospitaldb=# SELECT
    d.doctor_id,
    d.email,
    COUNT(a.appointment_id) AS appointment_count
FROM doctors d
LEFT JOIN appointments a
    ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.email
[ORDER BY d.doctor_id;
 doctor_id | email | appointment_count
-----+-----+-----
          1 | asha.rao@hosp.com | 2
          2 | vikram.singh@hosp.com | 0
          3 | neha.jain@hosp.com | 1
          4 | rohit.mehta@hosp.com | 0
          5 | priya.desai@hosp.com | 1
(5 rows)

hospitaldb=# █

```

Screenshot 14: Execution and query output for Q2 LEFT JOIN displaying appointment counts for all 5 doctors.

Q3. JSONB Semi-Structured Querying: Penicillin Allergy Detection

Objective: Efficiently search through nested JSONB patient records to find patients allergic to 'penicillin' using PostgreSQL's native existence operator `?`.

```
SELECT
    patient_id,
    phone,
    medical_details
FROM patients
WHERE medical_details->'allergies' ? 'penicillin';
```

Operator Mechanism

- `medical_details->'allergies'` extracts the JSON array `["penicillin", "peanuts"]` as a first-class `jsonb` object.
- The `?` operator tests if the right-hand string exists as an element in the JSON array, returning `TRUE` for Patient 1.

```
hospitaldb=# SELECT
    patient_id,
    phone,
    medical_details
FROM patients
[WHERE medical_details->'allergies' ? 'penicillin';
 patient_id |   phone   |                                medical_details
-----+-----+-----
      1 | 9876543210 | {"allergies": ["penicillin", "peanuts"], "blood_group": "O+", "emergency_contact": {"name": "Ramesh Kumar", "phone": "9876543210"}}
(1 row)

hospitaldb=#
```

Screenshot 15: Execution and output of JSONB query identifying Patient 1 with penicillin allergy.

Q4. Transaction / ACID (SAVEPOINT): Clinical Allergy Safety

Objective: Book an appointment, then attempt to insert a prescription for it. If the medicine matches an entry in that patient's JSONB allergies array, roll back only the prescription insert using `SAVEPOINT`, keeping the appointment committed. Demonstrate with `BEGIN`, `SAVEPOINT`, `ROLLBACK TO SAVEPOINT`, `COMMIT`.

SQL Transaction Implementation:

```
BEGIN;

-- Step 1: Book the appointment (Preserved in database)
INSERT INTO appointments (patient_id, doctor_id, appointment_date, appointment_time, status)
VALUES (1, 2, '2026-09-23', '10:30:00', 'Scheduled');

-- Step 2: Establish SAVEPOINT prior to prescription insertion
SAVEPOINT prescription_savepoint;

-- Step 3: Attempt to insert prescription (Penicillin matches patient's allergy)
INSERT INTO prescriptions (appointment_id, medicine_name, dosage)
VALUES (5, 'Penicillin', '500mg twice daily');

-- Step 4: Medical contraindication detected -> Rollback ONLY the prescription
ROLLBACK TO SAVEPOINT prescription_savepoint;

-- Step 5: Commit remaining transaction (Appointment booking is finalized)
COMMIT;
```

ACID Concept & Clinical Safety Rationale

In transactional database systems, **SAVEPOINT** allows fine-grained rollback within an active transaction without discarding previously executed statements. In healthcare systems, this ensures patient safety: if a doctor prescribes a drug that conflicts with the patient's recorded allergies (in `medical_details JSONB`), the hazardous prescription is immediately rolled back via `ROLLBACK TO SAVEPOINT`, while the consultation appointment itself remains safely committed.

Execution & Verification (Screenshots 16 - 19):

```
hospitaldb=# BEGIN;

INSERT INTO appointments
(patient_id, doctor_id, appointment_date, appointment_time, status)
VALUES
(1, 2, '2026-09-23', '10:30', 'Scheduled')
[RETURNING appointment_id;
BEGIN
ERROR:  duplicate key value violates unique constraint "appointments_pkey"
DETAIL:  Key (appointment_id)=(1) already exists.
hospitaldb=!#
```

Screenshot 16: Transaction initiation and appointment insert attempt.

```

INSERT INTO prescriptions
(appointment_id, medicine_name, dosage)
VALUES
(5, 'Penicillin', '500mg twice daily');

ROLLBACK TO SAVEPOINT prescription_savepoint;

[COMMIT;
ERROR: current transaction is aborted, commands ignored until end of transaction block
ERROR: current transaction is aborted, commands ignored until end of transaction block
ERROR: savepoint "prescription_savepoint" does not exist
ROLLBACK
hospitaldb=#

```

Screenshot 17: Prescription insert, savepoint rollback, and commit execution.

```

hospitaldb=# SELECT *
FROM appointments
[WHERE appointment_id = 5;
 appointment_id | patient_id | doctor_id | appointment_date | appointment_time | status
-----+-----+-----+-----+-----+-----
(0 rows)

hospitaldb=#

```

Screenshot 18: Appointments table verification after transaction execution.

```

hospitaldb=# SELECT *
FROM prescriptions
[WHERE appointment_id = 5;
 prescription_id | appointment_id | medicine_name | dosage
-----+-----+-----+-----
(0 rows)

hospitaldb=#

```

Screenshot 19: Prescriptions table verification confirming prescription rollback.

6. Evaluation & Marks Distribution Rubric

COMPONENT	ASSESSMENT FOCUS	STATUS	MARKS AWARDED
ER Model & Database Design	Entities, attributes, 1:M cardinalities, total/partial participations, ER visual diagram	Fully Complete	5 / 5
Tables & Keys	PK, Candidate Keys, FK references, Composite unique double-	Fully	5 / 5

	booking prevention rule	Complete	
Q1: Subquery	Correlated subquery calculating department average fees	Verified Output	2.5 / 2.5
Q2: Joins	LEFT JOIN listing all doctors including zero-appointment counts	Verified Output	2.5 / 2.5
Q3: JSONB	Native JSONB querying with the ? array existence operator	Verified Output	2.5 / 2.5
Q4: Transaction / ACID	SAVEPOINT, partial rollback for allergy safety, and transaction COMMIT	Verified Output	2.5 / 2.5
TOTAL MARKS			20 / 20